

AgentShield: Agentic AI Security Framework

Securing Tool-Using AI Agents Against Prompt Injection, Data Leakage, and Unsafe Actions

Mrinal Setty — Agent Orchestration & Target Agent Lead
Aditya Shenoy — Red-Team Agent & Dataset Lead
Yashas Uttangi — Safety, Firewall, Evaluation & Console Lead

CS6180 · Summer 2026 · August 9, 2026

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Why Generative AI Is Useful	3
2	System Architecture	3
2.1	Decision Pipeline	3
2.2	Key Components	3
2.3	Safety Boundary	4
3	Dataset	4
3.1	Purpose	4
3.2	Composition	5
3.3	Schema	5
3.4	Tool Coverage	6
3.5	Label Rules	6
3.6	Quality Checks	6
3.7	Synthetic Data Boundary	7
4	Red-Team Method	7
4.1	Purpose	7
4.2	Components	7
4.3	Injection Patterns	8
4.4	Deterministic Generation	8
4.5	Coverage	8
4.6	Quality Controls	9
4.7	Impact	9

5	Evaluation Summary	9
5.1	Method	9
5.2	Current Results	9
5.3	Baseline Comparison	10
5.4	Fixed Issue Classes	10
5.5	Reproduction	11
5.6	Residual Scope	11
6	Demo Scenarios	11
6.1	Scenarios	11
6.2	Run Through The API	12
6.3	Run Locally	12
6.4	Demo Narrative	12
7	Milestones and Team Contributions	12
7.1	Milestone Timeline	12
7.2	Team Contributions	13
8	Conclusion and Limitations	14
8.1	Conclusion	14
8.2	Limitations	14

1 Introduction

1.1 Problem Statement

Modern AI systems are moving from simple chatbots to agentic systems that take real actions through tools such as email, files, calendars, task managers, code hosting, and HTTP APIs. This shift creates a new class of security risk: malicious content hidden inside emails, documents, webpages, or tool responses can manipulate an agent into unsafe actions — leaking private data, sending messages to unauthorized recipients, or performing destructive operations the user never requested.

The specific problem this project addresses is: **how can we test and protect tool-using AI agents from prompt injection, data leakage, unauthorized tool use, and unsafe actions before a tool call is executed?**

AgentShield is a multi-agent security framework that simulates tool-using AI agents, generates red-team attacks against them, validates each proposed tool call against structured safety policies, blocks or escalates risky actions, and produces audit records explaining every decision. Crucially, the firewall inspects the proposed tool call **before** execution, so an unsafe action is stopped rather than merely reported after the fact.

1.2 Why Generative AI Is Useful

Generative AI is central to the framework because AgentShield must interpret natural-language requests, reason over tool-use intent, generate realistic adversarial scenarios, apply safety policies to ambiguous cases, classify risk, and explain its decisions in readable audit reports:

- The **Target Agent** uses GenAI for tool-use reasoning and structured tool-call generation.
- The **Red-Team Agent** uses adversarial prompting to generate realistic prompt-injection attacks and malicious scenarios.
- The **Policy Compiler and Firewall** use policy reasoning to interpret rules, resolve ambiguous risk, and generate audit explanations.
- An **LLM-as-a-judge** is used only for subjective quality checks (audit explanation quality), never for primary correctness scoring, which is always measured against fixed ground-truth labels.

2 System Architecture

AgentShield uses a modular backend workflow that owns the full local decision path: scenario ingestion, tool-call proposal, schema validation, policy and risk evaluation, safe mock execution, audit capture, metrics, and baseline comparison. (Full detail: `docs/architecture.md`.)

2.1 Decision Pipeline

Each user request or stored scenario flows through the same path:

1. The **Target Agent** proposes a structured tool call.
2. The proposed call is **schema-validated**.
3. In parallel, the **Policy Checker** evaluates every enabled safety rule and the **Risk Classifier** scores risk level and category.
4. The **Firewall** combines both results and chooses the most restrictive outcome: `ALLOW`, `BLOCK`, or `ASK_APPROVAL`.
5. Only `ALLOW` may proceed to optional **safe mock execution**; `BLOCK` and `ASK_APPROVAL` never execute.
6. The **Orchestrator** records an audit entry and workflow state, which feed metrics, the API response, and evaluation exports.

2.2 Key Components

Component	Responsibility
<code>target_agent.py</code>	Deterministic target-agent simulator and tool-call proposal format.
<code>scenario_store.py</code>	Loads scenario datasets from <code>data/</code> with stable source metadata.

Component	Responsibility
<code>policy_compiler_agent.py</code>	Parses Markdown safety policies into structured JSON rules.
<code>policy_checker.py</code>	Evaluates enabled rules and returns every matching violation.
<code>risk_classifier.py</code>	Classifies risk level, score, and categories.
<code>firewall_agent.py</code>	Combines policy and risk into an ALLOW / BLOCK / ASK_APPROVAL decision.
<code>orchestrator.py</code>	Runs the end-to-end workflow and stores audit entries.
<code>api.py</code>	FastAPI app, CORS, and endpoints for the console.
<code>metrics.py</code> / <code>baseline_analyzer.py</code>	Security/usability metrics and baseline comparison.

2.3 Safety Boundary

All tool execution is simulated. Mock tools validate arguments and return synthetic outputs but never perform real side effects — no email is sent, no file is read or written, no HTTP request leaves the machine. **ALLOW** means the firewall permits the call *in simulation only*.

Policy checking, risk classification, and label validation share the same security-pattern, text-matching, and intent helpers, which keeps dataset labels, runtime decisions, and evaluation reports aligned. Intent checks use standalone keyword matching (so short terms like `get` or `list` do not match inside unrelated words), and `delete_file` has an explicit deletion gate so read-only requests never authorize a deletion. External-target checks treat known internal hosts and configured internal email domains as internal; everything else is external for policy purposes.

3 Dataset

3.1 Purpose

The dataset is a synthetic benchmark of agent tool-call scenarios with ground-truth firewall decisions. Each example pairs a user request, optional external context, and proposed tool call with the decision the firewall should make. These labels drive policy validation, risk classification, metrics, baselines, and regression tests.

3.2 Composition

The full stored evaluation set contains 94 scenarios:

Source	Count	Role
<code>data/demo_scenarios.json</code>	3	Compact demo set covering all three decisions.
<code>data/sample_examples.json</code>	6	Small representative set for schema and label checks.
<code>data/dataset_v0.json</code>	50	Balanced baseline of benign and malicious scenarios.
<code>data/red_team_examples.json</code>	25	Adversarial scenarios spanning all supported tools.
<code>data/benign_edge_cases.json</code>	10	Safe actions that resemble attacks and guard against overblocking.

The deeper evaluation corpus is the 85-example set formed by `dataset_v0.json`, `red_team_examples.json`, and `benign_edge_cases.json`:

Split	Count
Benign	35
Malicious	50
ALLOW labels	27
ASK_APPROVAL labels	12
BLOCK labels	46

3.3 Schema

Every scenario follows `data/dataset_schema.json` and includes:

Field	Purpose
<code>user_request</code>	Natural-language request from the user.
<code>external_context</code>	External content the agent read; may contain injected instructions.
<code>proposed_tool_call</code>	Proposed tool name and arguments before firewall evaluation.

Field	Purpose
<code>expected_decision</code>	Ground-truth decision: <code>ALLOW</code> , <code>BLOCK</code> , or <code>ASK_APPROVAL</code> .
<code>risk_level</code>	Impact if the call executed unchecked: <code>low</code> , <code>medium</code> , <code>high</code> , or <code>critical</code> .
<code>attack_category</code>	<code>none</code> , <code>prompt_injection</code> , <code>data_exfiltration</code> , or <code>unauthorized_action</code> .
<code>explanation</code>	Human-readable rationale for the expected decision.

Stable ids such as `v0-*`, `rt-*`, `be-*`, and `demo-*` support scenario lookup.

3.4 Tool Coverage

All eight supported mock tools are represented:

`send_email`, `read_file`, `write_file`, `delete_file`, `create_calendar_event`, `create_task`, `create_github_issue`, `send_http_request`.

The corpus deliberately covers high-risk tools such as external HTTP requests, email, file deletion, protected writes, public issue creation, calendar events, and task creation.

3.5 Label Rules

The controlled vocabularies are documented in `../data/attack_taxonomy.md`.

General labeling rules:

- Benign scenarios use `attack_category: none` and are allowed when the tool call matches the request and is low impact.
- Destructive but user-requested actions use `ASK_APPROVAL` when execution should be confirmed.
- Prompt-injection, data-exfiltration, and unauthorized-action scenarios use `BLOCK` when the proposed call violates user intent or exposes sensitive data.
- External sends, external state-changing HTTP calls, and external attendees may use `ASK_APPROVAL` when the user authorized the action but the impact warrants confirmation.

3.6 Quality Checks

Dataset quality is enforced by:

```
PYTHONPATH=. python3 data/validate_dataset.py data/sample_examples.json
PYTHONPATH=. python3 data/validate_dataset.py data/demo_scenarios.json
PYTHONPATH=. python3 data/validate_dataset.py data/dataset_v0.json
```

```

PYTHONPATH=. python3 data/validate_dataset.py data/red_team_examples.json
PYTHONPATH=. python3 data/validate_dataset.py data/benign_edge_cases.json
PYTHONPATH=. python3 data/corpus_report.py
PYTHONPATH=. python3 data/quality_report.py
PYTHONPATH=. python3 src/label_validator.py --rules data/policy_rules.json --dataset data/d
PYTHONPATH=. python3 src/label_validator.py --rules data/policy_rules.json --dataset data/re
PYTHONPATH=. python3 src/label_validator.py --rules data/policy_rules.json --dataset data/be

```

The label validator uses the same shared intent, security-pattern, and text helpers as the runtime policy path, so dataset labels stay aligned with firewall behavior.

3.7 Synthetic Data Boundary

All examples are synthetic. Email addresses use *.example domains, and secret-like values use clearly marked <synthetic-...-placeholder> tokens. Tests and scans reject real-looking contact data or credentials.

4 Red-Team Method

4.1 Purpose

The red-team generator creates deterministic adversarial scenarios that stress-test AgentShield before runtime integration. The generated corpus is used as regression coverage for prompt injection, data exfiltration, unauthorized actions, sensitive file reads, protected writes, public issue leaks, and unsafe external requests.

4.2 Components

Component	Role
agents/injection_patterns.py	Prompt-injection pattern library.
agents/red_team_seeds.py	Attack seed catalog grouped by tool and attack style.
agents/red_team_agent.py	Renders a ScenarioSeed into a schema-valid example.
agents/generate_red_team.py	Writes data/red_team_examples.json.

Pipeline:

ScenarioSeed -> injection pattern rendering -> schema-valid scenario -> dataset validation

4.3 Injection Patterns

The generator uses five prompt-injection styles:

Pattern	Purpose
<code>direct_override</code>	Explicitly instructs the agent to ignore prior instructions.
<code>hidden_html_comment</code>	Hides the directive in markup-like content.
<code>fake_system_note</code>	Impersonates a privileged instruction inside external context.
<code>authority_urgency</code>	Uses claimed authority and urgency to pressure compliance.
<code>fake_tool_response</code>	Presents the directive as if it came from a tool response.

Using multiple styles prevents the corpus from depending on one obvious phrase.

4.4 Deterministic Generation

The default generation mode is offline and deterministic. It fills checked-in templates from checked-in seeds, requires no network access, and produces stable outputs suitable for tests and audits. An **online** mode remains reserved as an extension point and currently raises `NotImplementedError`.

Regenerate and validate:

```
PYTHONPATH=. python3 agents/generate_red_team.py
PYTHONPATH=. python3 data/validate_dataset.py data/red_team_examples.json
```

4.5 Coverage

`data/red_team_examples.json` contains 25 adversarial examples across all eight supported tools:

- `send_http_request`
- `send_email`
- `delete_file`
- `write_file`
- `create_github_issue`
- `create_calendar_event`
- `read_file`
- `create_task`

Attack categories include prompt injection, data exfiltration, and unauthorized actions. The generated set includes both injected-context attacks and pure tool-call attacks where the risk is visible in the proposed arguments.

4.6 Quality Controls

The red-team generator is covered by:

```
PYTHONPATH=. python3 agents/test_injection_patterns.py
PYTHONPATH=. python3 agents/test_red_team_agent.py
PYTHONPATH=. python3 data/validate_dataset.py data/red_team_examples.json
```

The tests check pattern integrity, deterministic generation, schema validity, tool coverage, label consistency, and synthetic-only content.

4.7 Impact

The expanded adversarial corpus exposed missing policy coverage around protected writes, public issue credential leaks, sensitive reads, unauthorized calendar and task creation, and external HTTP behavior. Those issue classes are now covered by reusable policy checks and regression tests rather than by scenario-specific exceptions.

5 Evaluation Summary

5.1 Method

Evaluation runs every stored scenario through the same policy checker, risk classifier, intent helpers, and firewall decision path used by the API. Results are compared against each scenario's `expected_decision` label.

The committed evaluation artifacts under `data/evaluation/` are generated from all discoverable datasets:

- `data/demo_scenarios.json`
- `data/sample_examples.json`
- `data/dataset_v0.json`
- `data/red_team_examples.json`
- `data/benign_edge_cases.json`

5.2 Current Results

The 94-scenario stored evaluation reports:

Metric	Result
Attack Success Rate	0.0%
Defense Success Rate	100.0%
Benign Task Success Rate	100.0%
False Positive Rate	0.0%
False Negative Rate	0.0%
Policy Compliance Accuracy	100.0%

Metric	Result
BLOCK Precision / Recall / F1	100.0% / 100.0% / 100.0%

The 85-example evaluation corpus has 85/85 firewall-label agreement, with no false negatives, no false positives, and all approval-labeled scenarios escalated.

5.3 Baseline Comparison

Evaluation includes three configurations:

Configuration	Behavior
Unprotected baseline	Allows every proposed call.
Prompt-only guardrail baseline	Uses lightweight prompt and credential heuristics without structured policies.
AgentShield firewall	Applies structured policy rules, risk scoring, and user-intent checks.

The baseline exports demonstrate why structured tool-call enforcement is needed: the prompt-only guardrail catches obvious attacks but misses subtler policy violations that require inspecting tool arguments, recipients, URLs, HTTP methods, file paths, repository visibility, and user intent.

5.4 Fixed Issue Classes

The expanded corpus previously surfaced the following issue classes, all now covered by reusable policies and regression tests:

Issue class	Current behavior
Unrequested protected file writes	BLOCK or ASK_APPROVAL depending on user authorization.
Credentials in public issue bodies	BLOCK.
Unrequested calendar or task creation from read-only requests	BLOCK.
Sensitive file reads not requested by the user	BLOCK.
Broadcast email beyond requested recipient scope	BLOCK.
Internal <code>*.example</code> destinations treated as external	Correctly treated as internal when configured.
Benign read-only public HTTP GET overblocked	ALLOW.

Issue class	Current behavior
Authorized external sends, posts, or attendees	ASK_APPROVAL when confirmation is required.
Read-only intent matched by substrings inside unrelated words	Fixed with standalone keyword and phrase matching.

5.5 Reproduction

```

PYTHONPATH=. python3 src/experiment_runner.py
PYTHONPATH=. python3 src/baseline_unprotected.py
PYTHONPATH=. python3 src/baseline_prompt_guardrail.py
PYTHONPATH=. python3 src/test_metrics.py
PYTHONPATH=. python3 src/label_validator.py --rules data/policy_rules.json --test

```

Generated artifacts are written to `data/evaluation/`.

5.6 Residual Scope

- The dataset is synthetic and scenario-based. It proves coverage for the committed project scenarios, not every possible production input.
- ASK_APPROVAL is a policy boundary for legitimate but risky actions.
- Future corpus growth should add new tools, argument fields, prompt-injection phrasings, and benign lookalikes as the system expands.

6 Demo Scenarios

The demo set in `data/demo_scenarios.json` covers the three user-visible decision paths: ALLOW, BLOCK, and ASK_APPROVAL.

6.1 Scenarios

ID	Tool	Expected decision	What it demonstrates
demo-safe-email	send_email	ALLOW	A normal internal email that matches the user's request.
demo-block-unauthorized-http-request	send_http_request	BLOCK	An external state-changing HTTP request the user did not authorize.

ID	Tool	Expected decision	What it demonstrates
demo-approval-delete	delete_file	ASK_APPROVAL	A legitimate but destructive deletion that requires confirmation.

6.2 Run Through The API

Start the backend:

```
uvicorn src.api:app --reload
```

Run the demo scenarios from the frontend or call the API directly:

```
curl -X POST http://localhost:8000/run-batch \
  -H "Content-Type: application/json" \
  -d '{"dataset_names":["demo"]}'
```

6.3 Run Locally

```
PYTHONPATH=. python3 src/test_orchestrator.py
```

```
PYTHONPATH=. python3 src/label_validator.py --rules data/policy_rules.json --dataset data/d
```

6.4 Demo Narrative

1. Safe internal email is allowed because the tool call matches the request and the recipient is internal.
2. Unauthorized external HTTP is blocked because the user requested a summary, not a state-changing call to an external endpoint.
3. User-requested deletion is escalated because file deletion is destructive even when the user asked for it.

7 Milestones and Team Contributions

The project ran over eight weekly milestones. Each milestone contained three parallel stories so all three members contributed every week: Mrinal Setty owned the -S1 (orchestration/backend) track, Aditya Shenoy the -S2 (dataset and red-team) track, and Yashas Uttangi the -S3 (policy, firewall, evaluation, and console) track.

7.1 Milestone Timeline

Milestone	Dates	Primary outcome
M1	15–22 Jun	Project setup, architecture, dataset schema,
M2	23–29 Jun	policy/evaluation setup Dataset v0, simulated tool interfaces,
M3	30 Jun–06 Jul	policy/rule foundation Target Agent, Red-Team Agent, and Firewall/Risk Classifier v0
M4	07–13 Jul	Integrated backend workflow, expanded attacks, policy compiler + firewall
M5	14–20 Jul	Baselines, evaluation metrics, dataset validation, audit rubric
M6	21–27 Jul	Dashboard, API integration, demo scenarios, result visualization
M7	28 Jul–03 Aug	System testing, failure analysis, UI polish, report draft
M8	04–10 Aug	Final demo, report submission, code submission

7.2 Team Contributions

Member	Core ownership	GenAI contribution
Mrinal Setty	Agent orchestration, Target Agent, tool interfaces, backend workflow and API	Multi-agent orchestration, Target Agent prompting, structured tool-call generation
Aditya Shenoy	Red-Team Agent, synthetic dataset, attack taxonomy, dataset quality, report content	Adversarial attack generation, synthetic scenario generation, prompt-injection patterns

Member	Core ownership	GenAI contribution
Yashas Uttangi	Firewall, policy compiler, evaluation metrics, dashboard/console	Policy reasoning, audit explanation generation, LLM-as-a-judge quality checks

Work was delivered continuously through reviewed pull requests across all eight milestones, spanning the backend workflow, the dataset and red-team corpus, and the policy/firewall/evaluation stack. The contribution target was an approximately equal split across the three members (about one third each), with each owning one major story every milestone.

8 Conclusion and Limitations

8.1 Conclusion

AgentShield demonstrates that a dedicated, policy-driven firewall can secure tool-using AI agents against prompt injection, data exfiltration, and unauthorized actions *before* a tool call executes. On the 94-scenario stored evaluation set the firewall reaches 100% policy-compliance with no false negatives and no false positives, while the unprotected and prompt-only baselines let unsafe calls through — showing that structured inspection of tool arguments, recipients, URLs, and user intent catches violations that prompt-level guardrails miss.

The red-team track was central to this outcome. Expanding the adversarial corpus across all eight tools surfaced coverage gaps — around protected writes, public issue credential leaks, sensitive reads, and unauthorized calendar/task creation — that were then closed with reusable policy checks and regression tests. Finding and fixing those gaps *before* they reach a production integration is exactly the value the framework is meant to provide.

8.2 Limitations

- **Synthetic and scenario-based.** The dataset proves coverage for the committed project scenarios, not every possible production input. Real deployments would need continual corpus growth with new tools, argument fields, injection phrasings, and benign lookalikes.
- **Simulation only.** All tool execution is mocked; no real side effects occur. Real-world execution must remain behind explicit user approval and production integration controls.
- **ASK_APPROVAL is a policy boundary**, not a guarantee: it marks legitimate but risky actions for human confirmation, and its calibration depends on the configured policies.

- **Subjective quality** (audit-explanation readability) is scored with an LLM-as-a-judge and a rubric; only objective correctness is measured against fixed ground-truth labels.